AMCJワークショップ
共鳴するコード、SuperColliderで創る音の世界
- 実践とその先へ

2026年3月14日

田所淳

00: 準備編

本日のワークショップの内容

書籍「共鳴するコード、SuperColliderで創る音の世界」のさらに先を紹介!

もちろん、SuperColliderの初歩からカバーしていきます。

環境設定

必要なアプリケーションをまずはインストールしていきましょう!

macOS / Windows どちらでもOK

- SuperCollider
- VScode
- Google Chrome

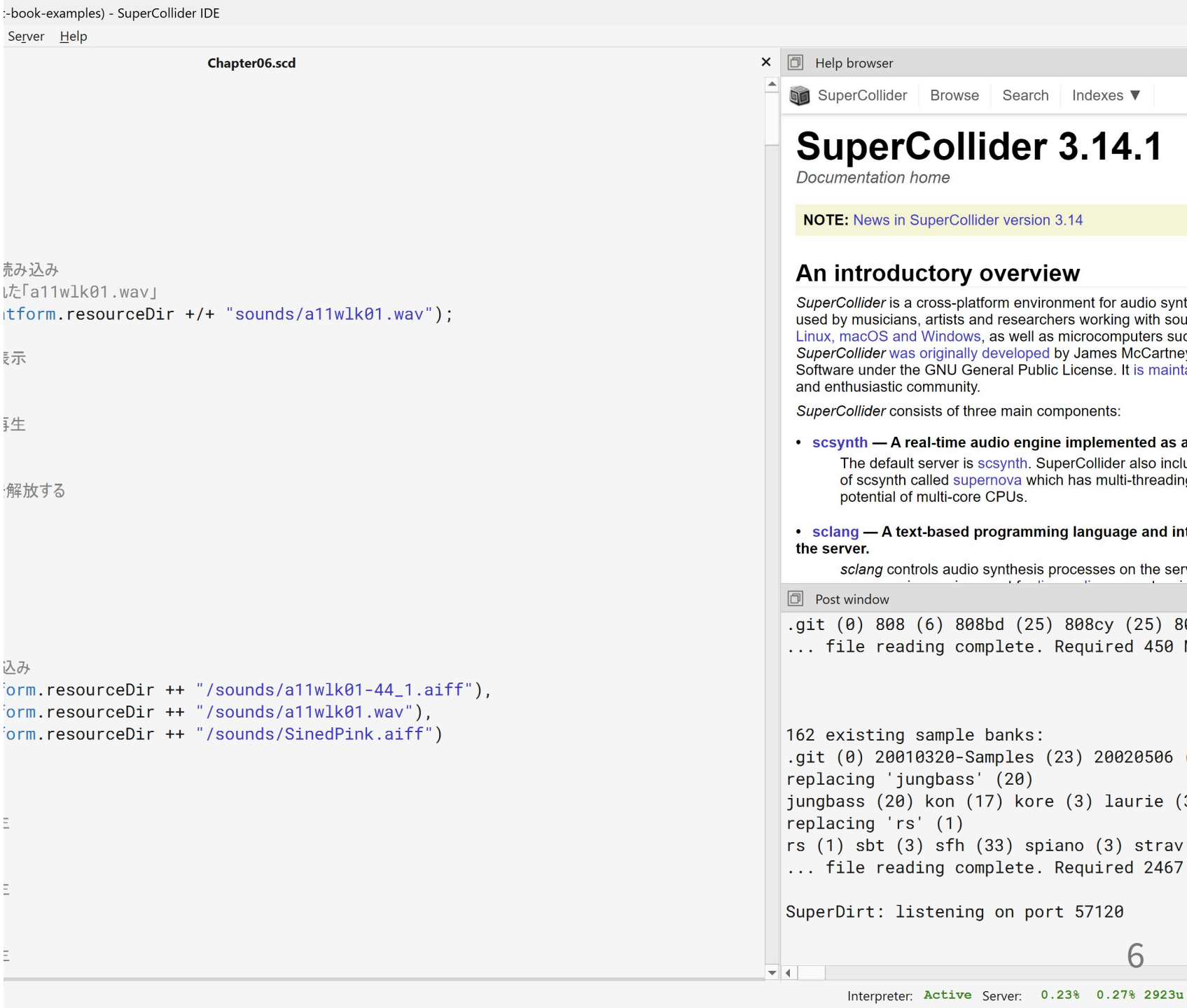
環境設定

本日のワークショップでは、コード生成のAIのアカウントがあるとなお便利です!

- Claude Code
- Github Copilot
- Gemini Code Assist
- ChatGPT codex

SuperCollider IDEの基本をおさらい

画面を操作しながら
解説していきます。



SuperCollider IDEの基本をおさらい

サーバー (scsynth) とクライアント (sclang) と開発環境 (IDE) の関係

本日の「その先」：1
VScodeからSuperColliderを使う

VScodeからSuperColliderを使う

SuperCollider IDEをVSCodeに置き換える

VScodeからSuperColliderを使う

SC IDEは便利ですが、VScodeを使用するとさらに
様々なメリットが

- コード補完
- シンタックスハイライト
- Git連携
- **AIコード生成ツールとの連携 ← !!**

Visual Studio

VScodeからSuperColliderを使う

VScode用のSuperColliderの拡張機能、
これまでもいくつか存在していた (ややこしい…)

- vscode-supercollider: Language support for the SuperCollider language
- language-supercollider: SuperCollider syntax grammars for VSCode
- scvsc: vscode as a supercollider client

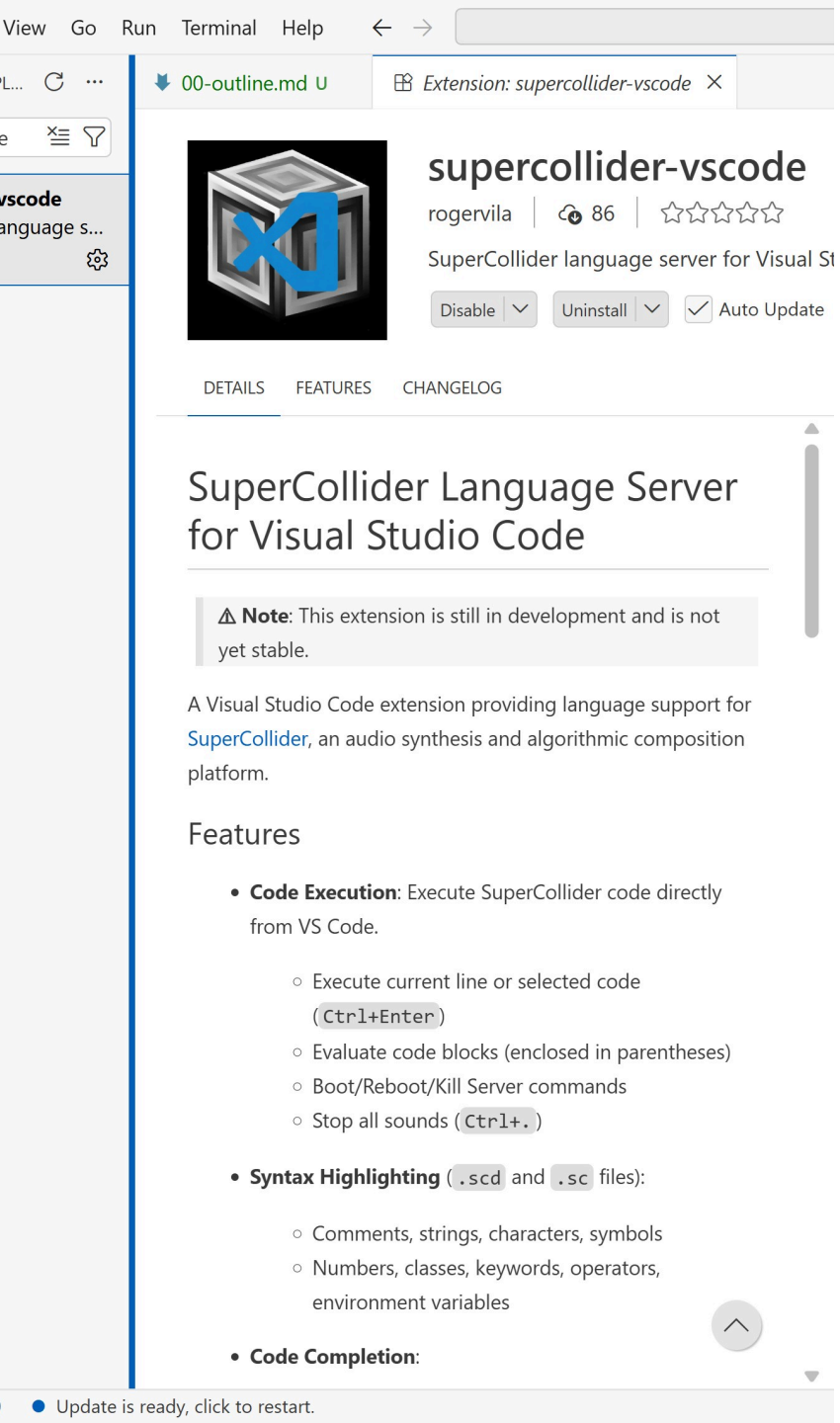
しかし、どれも一長一短
更新が止まっていたり、インストールが複雑だったり…

VScodeからSuperColliderを使う

しかし、最近新しい拡張機能が登場! とても便利、かつインストールが (比較的) 簡単!

supercollider-vscode (github)
SuperCollider language server for Visual Studio Code

- コード実行
- シンタックスハイライト
- コード補完
- エディター機能



VScodeからSuperColliderを使う

- VScodeの拡張機能
- “supercollider-vscode” で検索
- インストール

VScodeからSuperColliderを使う

環境設定: 以下のように、SuperColliderの言語であるsclangへのパスを通す必要があります。

macOS: 以下の記述を ~/.zshrc に追加

```
export PATH="$PATH:/Applications/SuperCollider.app/Contents/MacOS"
```

Windows: 環境変数のPATHに以下を追加

```
C:\Program Files\SuperCollider-3.14.1
```

VScodeからSuperColliderを使う

確認: 以下のコマンドがターミナルで実行できるか確認

MacOS: Terminal.appを開いて、以下のコマンドを実行

```
% sclang
```

Windows: PowerShellを開いて、以下のコマンドを実行

```
> sclang
```

VScodeからSuperColliderを使う

VScodeにはSuperCollider IDEのようなHelp Browserは無い
→ 公式ドキュメントのWeb版を活用しましょう!

- https://doc.sccode.org/

IDEのHelp Browserの中身と同じ内容がWeb上で閲覧できます。

AIコード生成を活用しながらSuperColliderをコーディング!!
最強の環境が実現しました!!

00-outline.md U ● | 01-sc-basic.scd ×

01-sc-basic.scd

```
69  // SECTION 3: 音量の変更
70  //
71  /*
72   * SinOsc.ar() の2番目の引数は「位相」、3番目の引数は「振幅（音量）」です。
73   *
74   * SinOsc.ar(freq, phase, mul, add)
75   *   freq  : 周波数（Hz）デフォルト: 440
76   *   phase : 位相（ラジアン）デフォルト: 0.0
77   *   mul   : 振幅（乗数）デフォルト: 1.0
78   *   add   : オフセット デフォルト: 0.0
79   *
80   * 注意: mul=1.0 はフルボリューム。0.0〜1.0 の範囲が適切です。
81   *   1.0を超えるとクリップ（歪み）が起きる可能性があります。
82   */
83
84  // 音量を0.3に下げる
85  { SinOsc.ar(440, 0.0, 0.3).dup(2) }.play
86
87  // 音量を0.8に上げる
88  { SinOsc.ar(440, 0.0, 0.8).dup(2) }.play
89
90  // キーワード引数を使った書き方（より明瞭）
```

Preview 00-outline.md | 01-sc-basic.scdの内容を教えてくだ...

01-sc-basic.scdの内容を教えてください。

01-sc-basic.scdの内容を教えてください。

c:\Users\tadok\Documents\GitHub\AMCJ26\01-sc-basic.scd

- *Thinking*
- **Read** `c:\Users\tadok\Documents\GitHub\AMCJ26\01-sc-basic.scd`
- 01-sc-basic.scd の内容は以下の通りです（全886行）。

構成

タイトル: Sin波から始めるSuperColliderの基本
12のセクションで構成されたチュートリアルファイルです：

セクション	内容
1	`SinOsc.ar()` — 最初の一音
2	`.dup(2)` / `! 2` — ステレオ再生
3	`mul` 引数 — 音量の変更
4	`freq` 引数 / `.midicps` — 周波数の変更
5	`var` / `~env` — 変数による指定
6	`SinOsc.kr`, `LFNoise1.kr`, `XLine.kr` — コントロール信号による変化（ビブラート、トレモロ等）

ctrl esc to focus or unfocus Claude

Edit automatically | 01-sc-basic.scd

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

Filter (e.g. text, !excludeText, t... | SuperCollider Post Win

```
compile done
localhost : setting clientID to 0.
internal : setting clientID to 0.

-> { SinOsc.ar(440, 0.0, 0.3).dup(2) }.play
Class tree inited in 0.41 seconds
server 'localhost' already booting
```

本日の「その先」の内容

- SuperColliderの基本をおさらい - 初歩からステップバイステップで
- 「その先」1: SuperColliderでvibe coding! - AIコード生成ツールを活用
- 「その先」2: Strudel + SuperColliderでライブコーディング!
- 「その先」3: WebブラウザーでSuperColliderを動かす! Super Sonic

ウォーミングアップ

以下のGithubリポジトリをZipでダウンロード
https://github.com/tado/AMCJ26

(もしAIエージェントが使用できる方は) 以下のプロンプトでAIを賢くしておきましょう!

> 「sc-book-examples」フォルダ内のChapter01からChapter08のSuperColliderのファイルの内容を学習してください。

Read `C:\Users\tadok\.claude\projects\c--Users-tadok`

Write `C:\Users\tadok\.claude\projects\c--Users-tadok`

16 lines

```
# AMCJ26 プロジェクト メモ

## プロジェクト概要
```

- Chapter 01〜08の学習が完了しました。各章の内容をま

Chapter 1 — サーバー設定（デバイス、バッファ、メモ
Chapter 2 — 基本言語：データ型、変数（var / `~env`）
Chapter 3 — UGenと波形：SinOsc、Pulse、Saw、Blip、
リアルタイム制御
Chapter 4 — ミキシングとエンベロープ：信号の加算、
再生
Chapter 5 — 合成手法：

- 加算合成（手動 → ループ → Klang → DynKlang）
- AM変調 / RM変調 / FM変調
- 減算合成（Saw + RLPF/RHPF/BPF）
- SuperSaw（MoogFFフィルター付き）

Chapter 6 — バッファとサンプル：Buffer.read、PlayBu
（TGrains / GrainBuf）
Chapter 7 — シーケンスとパターン：Routine + yield、
（\scale / \degree / \octave）
Chapter 8 — JITLib（ライブコーディング）：ProxySpac
接続、Pbindのホットスワップ

それでは、SuperColliderの「その先」の世界へ
飛び込んでいきましょう！